BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI – HYDERABAD CAMPUS

ME F320 Engineering Optimization · Course Project · Midsem Report · First Semester 2026–27

Thermal-Aware Workload Allocation in AI Data Centres: A Linear Programming Approach with Dynamic Thermal State

[Student 1 name] (Roll No. _____) [Student 2 name] (Roll No. _____)

Submitted: 13 October 2026

1. Introduction

AI data centres are among the fastest-growing consumers of electricity, and cooling is a large share of that demand; Lu and Wang report it as roughly 35 % of data-centre energy [1]. Raising the inlet-air set-point of the server room reduces cooling energy, and both industry guidance and regulation now push in that direction [2, 3]. The difficulty, demonstrated experimentally in [1], is that a warmer room removes heat from GPUs less effectively: under high-intensity LLM inference, the heat generated can exceed the heat removed, the GPU crosses its thermal limit and throttles, and throughput and energy efficiency both suffer.

The reference work [1] addresses this with TAWS, a reinforcement-learning scheduler that jointly selects job placement, batch size and per-GPU voltage–frequency settings. This project keeps the same physical problem but isolates the placement decision and formulates it as a linear program (LP). The LP gives up the voltage–frequency control of [1] in exchange for three things a learned policy cannot provide: a certificate of global optimality, an exact feasibility test (whether a given set-point can be operated safely at all), and dual prices that quantify how much energy one extra degree of thermal headroom is worth.

Since the outline report, we have implemented both models planned there, validated them, and obtained preliminary results. The static model (M1) from the outline is retained; the dynamic model (M2) has been refined to use an exact discretisation of the thermal dynamics, and a set-point analysis that includes cooling energy has been added as an extension beyond [1].

2. Background and Significance of the Study

Thermal throttling. GPU power is converted to heat in the die; heat generation depends on core voltage and frequency and is well approximated by electrical power through the first law [1]. If generation exceeds removal, the die temperature rises until it reaches a threshold at which firmware lowers voltage and frequency to protect the hardware. In the motivating experiment of [1], an air-cooled RTX 4090 running Llama-3-8B entered throttling about 1.5 % of the time at a 28 °C inlet but about 15.3 % of the time at 41 °C, with throughput losses of up to 34.2 %.

Cooling regulation. ASHRAE guidance recommends an inlet temperature of about 27 °C [2], the EU Code of Conduct suggests around 35 °C to avoid overcooling [3], and [1] cites work showing that operating at up to 41 °C can maximise energy savings without serious hardware-lifetime impact [5]. Each degree is reported to save several percent of cooling energy; [1] cites roughly 8 % per °C and uses 5 % per °C in its own model.

Thermal-aware scheduling. Existing schedulers optimise throughput, utilisation, SLOs or carbon but implicitly assume cooling is always sufficient [1]; thermal- and power-aware placement for LLM inference has recently been studied in production cloud settings [4]. These approaches are heuristic or learned.

Significance. An LP formulation is attractive here for reasons beyond convenience. The feasible set is a convex polyhedron, so the simplex method returns a global optimum [6, 7]. Infeasibility is detected exactly, which turns ‘what is the highest safe set-point?’ into a well-posed computation. Duality attaches an economic value to every constraint: the dual of a thermal constraint is the marginal energy cost of the temperature limit, which is precisely the quantity a data-centre operator trades against cooling energy when choosing a set-point.

3. Problem Statement and Objectives

A data centre contains a set J of heterogeneous GPUs and serves a set I of inference workload types whose demand varies over a planning horizon. GPUs differ in compute capacity, energy per unit of work, idle power, cooling conductance, thermal mass and throttling threshold. **Determine how much of each workload to run on each GPU at each time step so that total GPU energy is minimised, all demand is served, no GPU exceeds its capacity, and no GPU reaches its throttling temperature at any time, for a given server-room set-point T_{amb}. A secondary question is how the minimum energy and the feasibility of the problem change as T_{amb}** is raised.

Objectives (status at midsem in brackets):

- O1. Formulate and solve the static allocation LP M1 [done].
- O2. Formulate and solve the multi-period LP M2 with GPU temperature as a decision variable [done; discretisation refined to exact zero-order hold].
- O3. Validate both models against hand solutions, independent solvers, duality theory, analytical limits and the physics of the thermal model [done, Section 7].
- O4. Compare against thermal-agnostic and myopic thermal-aware baselines on energy and thermal-limit violations [done, preliminary].
- O5. Duality and sensitivity analysis: shadow prices of thermal headroom; parametric sweep of T_{amb} [done, preliminary]; cost-coefficient ranging [final report].
- O6. Extension beyond [1]: determine the energy-optimal set-point when cooling energy is included [preliminary]; add temperature-dependent leakage power and demand uncertainty [final report].

4. Assumptions

- A1. **Divisible work.** Each workload is a continuous stream that may be split across GPUs in any proportion (fluid approximation of a large number of small inference jobs). This is what makes the problem an LP rather than an integer program.
- A2. **Linear power model.** GPU power is $P_{jt} = P^{idle}_{j} + \Sigma_i \gamma_{ij} x_{ijt}$: idle power plus energy per unit of work times rate. Voltage–frequency is fixed at its default setting (no DVFS), unlike [1].
- A3. **Heat equals power.** All electrical power is dissipated as heat in the GPU (first-law approximation used in [1], Eq. 2).
- A4. **Lumped thermal model.** Each GPU and its heat sink form one thermal mass $m_j c_j$ cooled to the inlet air through a constant conductance k_j (Newton's law of cooling) [12]; GPUs do not heat one another and T_{amb} is uniform and constant over the horizon.
- A5. **Hard thermal limit.** The LP keeps every GPU at or below $T^{thr}_{j} - \delta$ with a safety margin $\delta = 2$ °C; throttling is therefore never triggered by an LP solution.
- A6. **Known demand.** Demand D_{it} is known over the 30-minute horizon (deterministic look-ahead). Uncertainty is deferred to the final report.
- A7. **All GPUs on.** Idle power is paid by every GPU, so it is a constant in the objective; switching GPUs off would need binary variables (MILP) and is out of scope.
- A8. **Synthetic parameters.** All numerical parameters (Table 1) are synthetic but chosen to be physically plausible (full-load power 300–450 W, thermal time constants of about 2–3 minutes). They are not measurements of real hardware.
- A9. **Cooling energy (extension only).** CRAC energy is proportional to the heat removed, equal to 0.6 × GPU energy at 25 °C (consistent with the 30–40 % cooling vs 50–60 % IT shares quoted in [1]) and falling geometrically by $s = 5$ % or 8 % per °C of set-point increase [1].

5. Mathematical Modelling

5.1 Nomenclature

Symbol	Meaning (units)
$i \in I, j \in J, t = 0..N-1$	workload type, GPU, time step of length $\Delta t = 10$ s ($N = 180$, i.e. 30 min)
x_{ijt} (or x_{ij} in M1)	**decision:** rate of workload i served on GPU j in step t (work units/s)

Symbol	Meaning (units)
T_{jt}	**decision (M2):** temperature of GPU j at the end of step t (°C)
D_{it}, C_j	demand (units/s); capacity (units/s)
γ_{ij}	energy per unit of workload i on GPU j (J/unit); equals heat per unit by Assumption 3
P^{idle}_j, k_j, $m_j c_j$	idle power (W); cooling conductance (W/°C); thermal capacitance (J/°C)
T^{thr}_j, δ, T_{amb}	throttling threshold (°C); safety margin (°C); inlet-air set-point (°C)
$a_j = exp(-k_j \Delta t / m_j c_j)$	exact one-step decay factor of the thermal model (dimensionless)

5.2 Model M1 — steady-state allocation LP

If a GPU dissipates constant power P_j, its temperature settles at $T_{amb} + P_j/k_j$. Requiring this steady state to stay below $T^{thr}_j - \delta$ gives a heat budget $Q_j = k_j(T^{thr}_j - \delta - T_{amb}) - P^{idle}_j$ on the dynamic power. This replaces the arbitrary budget H^{max}_j of the outline with one derived from the physics, and links M1 to T_{amb}. M1 is:

$$\min \; Z_1 = \Sigma_i \Sigma_j \gamma_{ij} x_{ij} \tag{1}$$

$$\Sigma_j x_{ij} = D_i \quad \forall i ; \qquad \Sigma_i x_{ij} \le C_j \quad \forall j \tag{2, 3}$$

$$\Sigma_i \gamma_{ij} x_{ij} \le Q_j = k_j(T^{thr}_j - \delta - T_{amb}) - P^{idle}_j \quad \forall j ; \qquad x_{ij} \ge 0 \tag{4, 5}$$

Because γ_{ij} depends on the workload as well as the GPU, constraint (4) is not a multiple of (3), so the thermal and capacity limits act independently (the redundancy issue identified in the outline does not arise).

5.3 Model M2 — multi-period LP with thermal state

M1 assumes every GPU sits at steady state, which ignores the thermal mass that lets a GPU absorb a short burst of heat. M2 relaxes this assumption by making temperature a decision variable. The lumped model $m_j c_j \, dT_j/dt = P_j - k_j(T_j - T_{amb})$ is integrated exactly over a step during which P_{jt} is constant (zero-order hold), giving

$$T_{j,t} = a_j \, T_{j,t-1} + (1 - a_j) \, [\, T_{amb} + (P^{idle}_j + \Sigma_i \gamma_{ij} x_{ijt}) / k_j \,] \tag{6}$$

The outline proposed an explicit Euler step; the exact form (6) is equally linear, has no discretisation error for piecewise-constant power and is stable for any Δt (Section 7, V6). M2 is:

$$\min \; Z_2 = \Sigma_t \Sigma_i \Sigma_j \gamma_{ij} \Delta t \, x_{ijt} \quad (+ N \Delta t \, \Sigma_j P^{idle}_j, \text{ constant }) \tag{7}$$

$$\Sigma_j x_{ijt} = D_{it} \;\; \forall i,t ; \qquad \Sigma_i x_{ijt} \le C_j \;\; \forall j,t ; \qquad (6) \; \forall j,t \;\; \text{with } T_{j,-1} = T^0_j \text{ given} \tag{8–10}$$

$$T_{jt} \le T^{thr}_j - \delta \quad \forall j,t ; \qquad x_{ijt} \ge 0 \tag{11, 12}$$

5.4 Why both models are linear programs

In (1)–(12) the parameters γ_{ij}, C_j, D_{it}, k_j, a_j, P^{idle}_j, T_{amb} and T^{thr}_j are constants. Every decision variable (x, T) appears only to the first power and no two decision variables are multiplied; in particular, the exponential in a_j involves only constants. All variables are continuous. The objective is therefore linear and the feasible set is an intersection of half-spaces and hyperplanes, i.e. a convex polyhedron, so both problems are LPs and any optimum found by the simplex method is global [6, 7]. The dual variable of (11) has units of J/°C: it is the reduction in optimal energy obtained by relaxing the temperature limit of GPU j at step t by one degree. For the instance in Section 6, M2 has 7,560 variables and 3,240 constraints plus bounds.

6. Solution Methodology

Instance. Six GPUs (air- and water-cooled consumer-class cards loosely modelled on the RTX 3090 / 4090 pair used in [1], one data-centre-class accelerator and one older card) serve six LLM inference workload types (prefill-heavy, chat decode, summarisation, code generation, embedding, long-context RAG). Energy coefficients combine a GPU efficiency, a workload intensity and an affinity term (compute-bound work is cheaper on the newer consumer cards, memory-bound work on the high-bandwidth accelerator). Total demand follows a slow wave with two short bursts that reach 97 % of total capacity (Fig. 3, top). Initial temperatures are T_{amb} + 25 °C. Parameters are listed in Table 1.

Table 1. GPU parameters (synthetic). $\tau = m_j c_j / k_j$ is the thermal time constant.

GPU	C_j (u/s)	P^{idle}_j (W)	k_j (W/°C)	$m_j c_j$ (J/°C)	τ (s)	T^{thr}_j (°C)	γ_{ij} range (J/u)
G1 4090-AC	1.6	30	7.2	1100	153	83	161–233
G2 4090-WC	1.6	32	12.0	2000	167	83	164–238
G3 3090-AC	1.0	25	6.0	900	150	83	227–330
G4 3090-WC	1.0	27	10.0	1600	160	83	231–336
G5 A100-class	2.0	50	8.5	1400	165	85	113–219
G6 Legacy-AC	0.8	30	5.6	750	134	83	287–418

Solvers. Both models are built as sparse matrices in Python and solved with SciPy's linprog routine [8] using the HiGHS dual revised simplex method [9]. Each result is re-solved with the HiGHS interior-point method and with CVXOPT [10], an independent primal–dual interior-point code; the thermal-free version of M1 is additionally solved as a min-cost flow with the network simplex method in NetworkX [11]. Dual values are taken from HiGHS and checked against the primal objective.

Baselines. Three heuristics are simulated with the same thermal model (6): (B1) **greedy lowest-energy**, which fills the cheapest workload–GPU pairs first and ignores temperature; (B2) **capacity-proportional**, which splits every workload across GPUs in proportion to capacity, as a simple load balancer would; and (B3) **myopic thermal greedy**, which is greedy but limits each GPU's power so that its temperature at the end of the current step stays below the limit, i.e. thermal-aware with a one-step look-ahead, loosely analogous to the reactive baselines of [1]. When B3 cannot place all demand within the limits it places the remainder on the GPU with the most spare capacity and records it as forced work. For the baselines a 'violation' is GPU-time spent above T^{thr}_j; we do not model the throughput loss that throttling would cause, so baseline energies are optimistic.

7. Validation

Validation was completed before generating results. Table 2 summarises nine checks; three are worth explaining.

V1 (hand solution). Two GPUs and two workloads with D = (1.0, 1.2), C = (1.5, 1.5), γ = [200 260; 150 220] and heat budgets Q = (220, 400) W. Using the demand equalities to eliminate x_{12} and x_{22} leaves a two-variable problem that can be solved graphically (Fig. 1). Workload 2 saves 70 W per unit by moving to G1 but adds only 150 W of heat (0.47 W saved per W of heat), against 0.30 for workload 1, so G1's heat budget is spent on workload 2 first. The optimum is x_{21} = 1.2, x_{11} = 0.2 with Z = 428 W, identical to the simplex result. The thermal dual of G1 is −0.30 W/W and the demand duals are 260 and 195 W per unit/s, both derivable by hand from the same marginal argument.

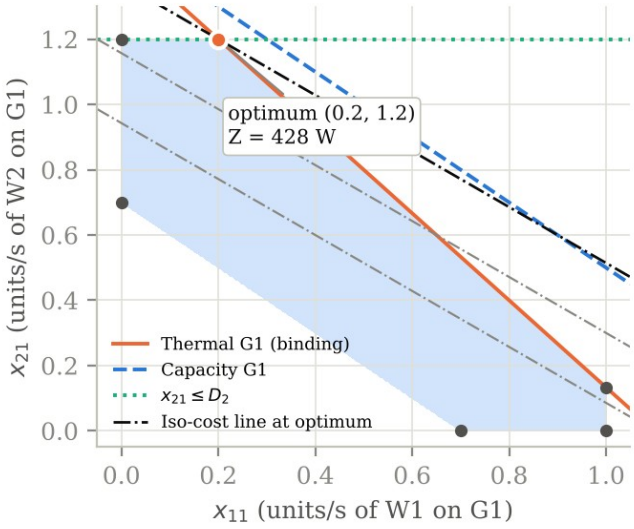


Fig. 1. Feasible region (shaded) of the hand-solved instance in the reduced variables (x_{11}, x_{21}). The optimum lies at the intersection of G1's thermal constraint and $x_{21} \le D_2$; G1's capacity constraint is slack.

V4 (shadow prices). The thermal dual is the most important output of the duality analysis, so it was checked directly: raising T^{thr} of the binding GPU by 0.05 °C and re-solving changes the optimum by the amount the dual predicts, to four significant figures in M1 and within 0.05 % in M2 (the small difference is because the perturbation changes which constraints are active at a few time steps).

V6 (thermal model). The exact zero-order-hold update agrees with the analytical solution of the ODE to machine precision, whereas the Euler step proposed in the outline is in error by 0.68 °C at Δt = 10 s. Since the safety margin is only 2 °C, this justifies the change of discretisation.

Table 2. Validation checks.

Check	Result	
V1 Hand-solved 2×2 instance (graphical method / vertex enumeration vs simplex)	Optimum (0.2, 1.2), Z = 428 W by both; duals match hand derivation (0.30, 260, 195)	Pass
V2 Cross-solver agreement: HiGHS simplex, HiGHS IPM, CVXOPT; M1 and M2 at 35 and 41 °C	max. relative deviation 3.9×10^{-9}	Pass
V3 Strong duality: primal vs dual objective	max. relative gap 1.0×10^{-15}	Pass
V4 Shadow price vs finite difference (+0.05 °C on the binding GPU, 41 °C)	M1: -2.7727 vs -2.7727 W/°C; M2: -3436 vs -3434 J/°C	Pass
V5 Thermal limits removed ⇒ transportation problem; network simplex	988.046 vs 988.050 W (difference from integer scaling, 3.2×10^{-6})	Pass
V6 Thermal model vs analytical solution of the ODE (constant 430 W on G1)	exact ZOH error ≤ 1.3×10^{-13} °C; Euler error 0.68 °C at Δt = 10 s, 2.16 °C at 30 s; steady state 94.72 °C reached	Pass
V7 Identical GPUs, slack thermal limits: $Z = \Sigma_i D_i\gamma_i$	850.0 vs 850.0 W	Pass
V8 Infeasibility detection (T_{amb} = 70 °C)	solver status ‘infeasible’ as expected	Pass
V9 LP temperatures re-simulated independently	max. difference 5.0×10^{-13} °C	Pass

8. Preliminary Results

8.1 Static allocation (M1)

At T_{amb} = 41 °C and mean demand, M1 draws 1204.9 W in total against 1182.0 W when thermal limits are ignored, so thermal safety costs 1.9 % extra power. Fig. 2 shows how the allocation changes: the efficient but poorly cooled G1 (4090-AC) is held at 82 % utilisation by its thermal constraint, prefill work moves to its water-cooled twin G2, and summarisation spills onto the less efficient G3. The thermal dual of G1 is 0.385 W per W of heat budget, i.e. 2.77 W per °C of headroom; G2 and G5 are capacity-bound (capacity duals 80.5 and 110.0 W per unit/s), and no other thermal constraint is binding.

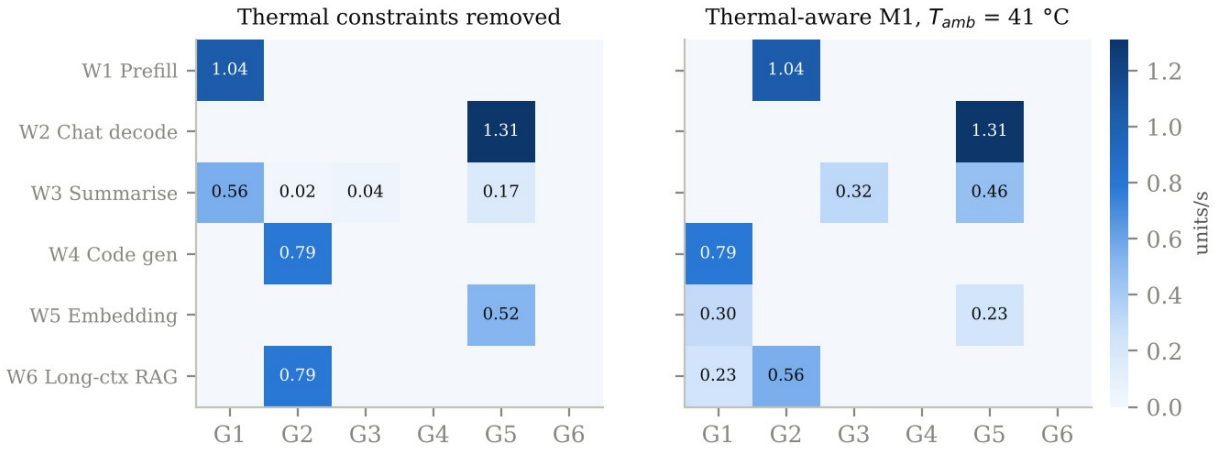


Fig. 2. Optimal M1 allocation (units/s) at 41 °C, without (left) and with (right) thermal constraints.

8.2 Dynamic allocation (M2) against baselines

Table 3 compares M2 with the three baselines over the 30-minute horizon. At both set-points the LP keeps every GPU below the limit, and at 35 °C it is also the cheapest method overall. The thermal-agnostic greedy heuristic is within 0.5 % of the LP on energy but spends 13.7 % (35 °C) and 15.3 % (41 °C) of GPU-time above the throttling threshold, peaking at 92.6 °C; in practice these intervals would

throttle and lose throughput. Capacity-proportional balancing uses about 18 % more energy because it places work on inefficient GPUs, and it still overheats the poorly cooled cards at 41 °C. The myopic thermal greedy heuristic is safe at 35 °C at a 0.78 % energy penalty, but at 41 °C it runs out of headroom during the bursts, must force 122 unit·s of work onto hot GPUs and exceeds the limit.

Table 3. M2 versus baselines over 30 min. 'Over limit' is the share of GPU-time above T^{thr}; 'Excess' is the time integral of the exceedance.

Method	Energy (MJ)	vs LP	Over limit (%)	Peak (°C)	Excess (°C·s)	Forced (u·s)
Set-point T_{amb} = 35 °C						
LP (M2)	2.1722	—	0.0	81.0	0	0
Greedy lowest-energy	2.1790	+0.31 %	13.7	86.6	3889	0
Capacity-proportional	2.5807	+18.81 %	0.5	84.2	30	0
Myopic thermal greedy	2.1891	+0.78 %	0.0	81.0	0	0
Set-point T_{amb} = 41 °C						
LP (M2)	2.1888	—	0.0	81.0	0	0
Greedy lowest-energy	2.1790	-0.45 %	15.3	92.6	13361	0
Capacity-proportional	2.5807	+17.91 %	3.9	90.2	1138	0
Myopic thermal greedy	2.2119	+1.06 %	0.7	86.1	143	122

Fig. 3 shows why look-ahead matters. Under greedy allocation G1 settles near 93 °C and stays above the threshold. The LP instead holds G1 at its limit and, shortly before each burst (about 8 min and 20 min), deliberately lowers G1's load so that it cools by a few degrees; that stored headroom is then used during the burst. This anticipatory pre-cooling cannot be produced by a one-step-ahead rule, which is why B3 fails at 41 °C while M2 does not.

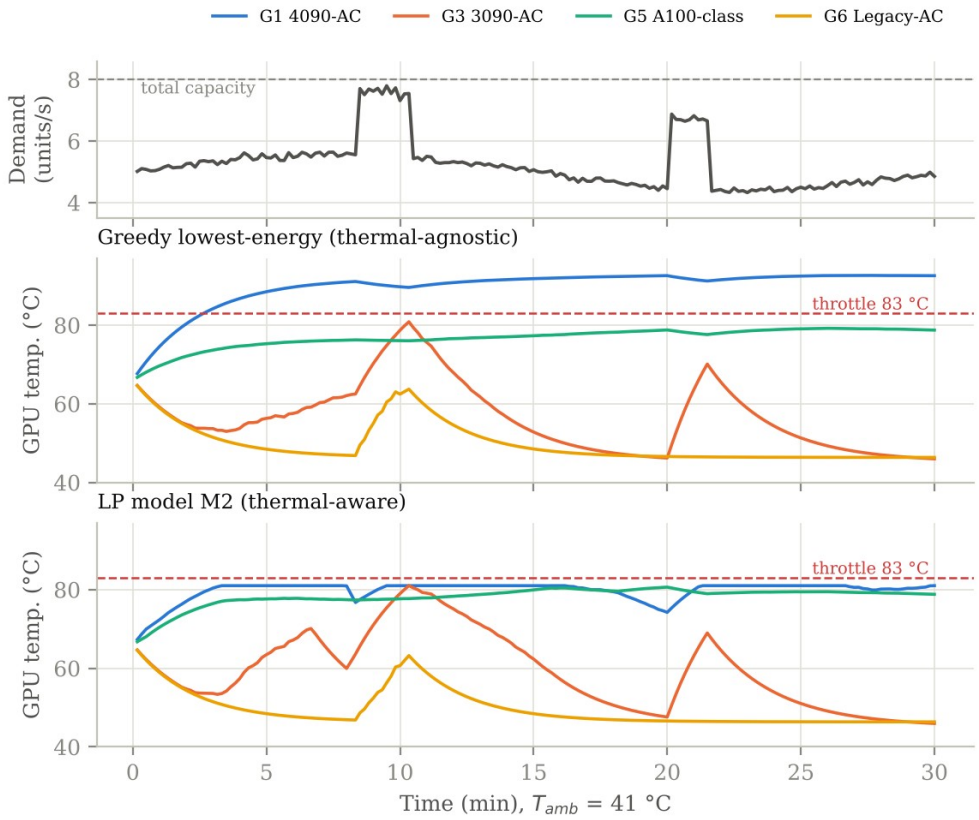


Fig. 3. Demand (top) and temperatures of four GPUs under greedy allocation (middle) and LP model M2 (bottom) at 41 °C.

8.3 Effect of the ambient set-point

Fig. 4 repeats the comparison for set-points from 25 to 50 °C, mirroring the throttling-versus-ambient experiment in [1, Fig. 2(a)]. Greedy allocation first exceeds the limit at 32 °C, capacity-proportional at 34 °C and the myopic heuristic at 40 °C; the LP never does. The LP also gives an exact operating limit: M2 becomes infeasible above 47.7 °C, meaning no allocation of this demand can avoid throttling. The static model M1 applied to peak demand is infeasible above 36.3 °C, so relaxing the steady-state assumption raises the safe set-point by about 11 °C for this instance. The rise in the greedy curve from about 13 % to 15 % between 35 and 41 °C is numerically close to the 15.3 % throttling share measured in [1] at 41 °C; since our parameters are synthetic, we regard this as a coincidence rather than validation.

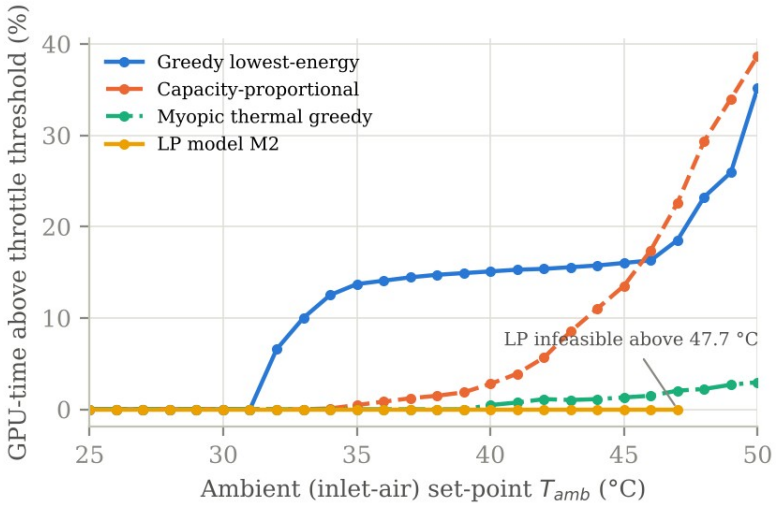


Fig. 4. Share of GPU-time above the throttling threshold versus set-point.

8.4 Extension: energy-optimal set-point including cooling

Reference [1] fixes the set-point and maximises throughput. As an extension we ask which set-point minimises facility energy (GPU plus cooling) when allocation is optimal, solving M2 at each set-point (a parametric LP) and adding cooling energy as in Assumption 9. Fig. 5(a) shows that optimal GPU energy rises only slowly with T_{amb} (1.1 % from 25 to 41 °C) because the LP shifts work to better-cooled GPUs rather than less efficient ones until close to the limit. Cooling savings dominate (Fig. 5(b)), so under these assumptions facility energy decreases monotonically and the optimal set-point is the highest one the LP can still serve safely. Moving from the M1 limit (36 °C) to 41 °C saves 5.1–6.0 % of facility energy for s = 5–8 % per °C; going all the way to the M2 limit (47 °C) would save 9.0–9.7 %. Set-points above 41 °C lie outside the range tested in [1] and our model omits leakage power, which grows with temperature and would penalise hot operation, so the higher figure should be treated as an upper bound.

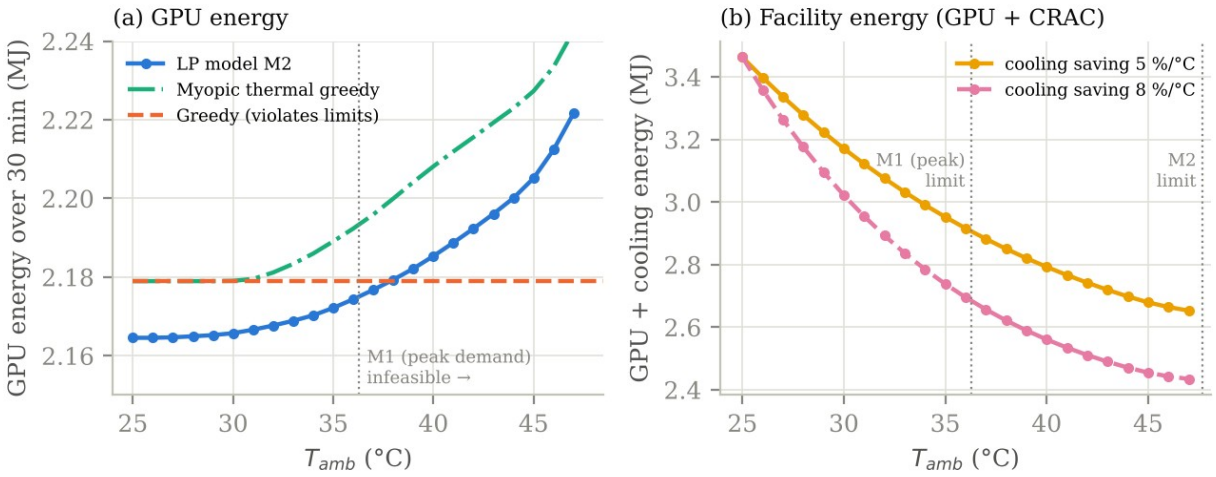


Fig. 5. (a) GPU energy versus set-point for the LP and two heuristics; greedy is shown for reference only since it violates the limits. (b) GPU plus cooling energy for two cooling-saving rates.

8.5 Shadow prices of thermal headroom

Fig. 6 plots the summed dual of constraint (11) for each GPU against T_{amb}, i.e. how much 30-minute energy one extra degree of headroom on that GPU would save. Only G1 has a material price below 43 °C, rising from zero at 26 °C to about 3.4 kJ per °C at 41 °C; the A100-class card becomes the bottleneck only above 44 °C. For an operator this identifies where cooling upgrades pay off: improving G1's heat sink is worth something at any realistic set-point, while upgrading the water-cooled cards is worth nothing below about 47 °C.

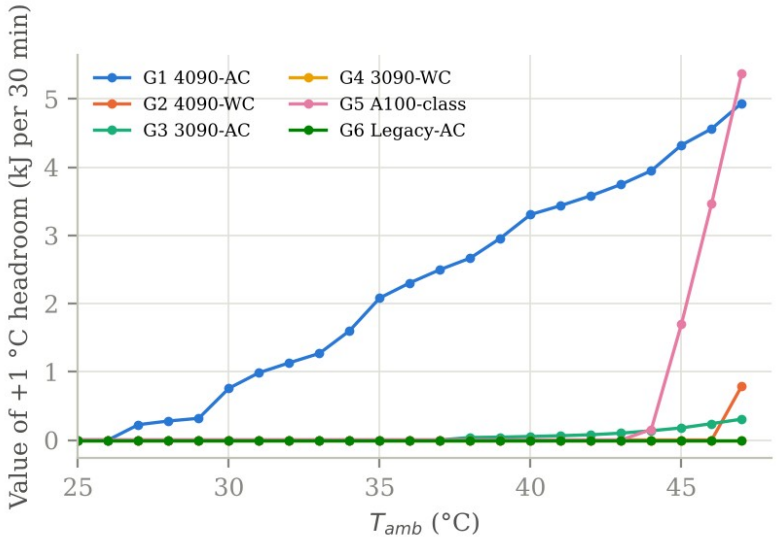


Fig. 6. Value of one extra degree of thermal headroom per GPU (negative of summed dual of constraint (11)).

Computation. M2 solves in 0.15 s for the 30-minute horizon (7,560 variables) and 1.16 s for 2 hours (30,240 variables), so re-solving every few seconds in a rolling-horizon scheduler would be practical.

9. Discussion

What the LP adds relative to [1]. The reference scheduler is evaluated empirically and cannot state whether a better schedule exists or whether a set-point is operable at all. Our model answers both for the placement sub-problem: the optimum is certified by strong duality (V3), infeasibility marks the highest safe set-point (Section 8.3), and the duals price thermal headroom per GPU (Section 8.5). The qualitative behaviour matches [1]: thermal-agnostic allocation overheats increasingly as the room is warmed, and thermal-aware allocation removes the violations at a small energy cost.

How the work goes beyond the reference and the outline. (i) The steady-state assumption of M1 was relaxed by modelling thermal capacitance in M2, which raises the safe set-point by about 11 °C and

produces anticipatory pre-cooling. (ii) The heat budget is now derived from the cooling physics rather than assumed. (iii) The discretisation was made exact. (iv) A facility-level set-point analysis including cooling energy was added, which [1] does not perform.

Limitations. The parameters are synthetic, so absolute numbers (energies, limits, percentages) are illustrative and should not be compared quantitatively with [1]. Baseline energies ignore throttling losses and are therefore optimistic for the baselines. There is no DVFS, unlike [1], so the LP cannot trade clock speed for temperature. Leakage power, GPU-to-GPU heat coupling and variation of T_{amb} during the horizon are neglected, and demand is assumed known.

Plan for the final report. (1) Add temperature-dependent leakage $\beta_j(T_{jt} - T_{ref})$ to the power model; it is affine in T, so M2 remains an LP, and it should create an interior energy-optimal set-point. (2) Make T_{amb} a decision variable with a convex piecewise-linear cooling-energy curve. (3) Cost-coefficient ranging on γ_{ij} and right-hand-side ranging on C_j. (4) A scenario-based treatment of demand uncertainty. (5) A throughput-maximising variant closer to the objective of [1], and a rolling-horizon test in which the LP is re-solved as demand is revealed.

References

[1] R. Lu and D. Wang, "A thermal-aware workload scheduler for high-performance LLM inference in cooling-regulated datacenters," ACM SIGENERGY Energy Informatics Review, vol. 5, no. 2, Jul. 2025.

[2] ASHRAE Technical Committee 9.9, Thermal Guidelines for Data Processing Environments, 5th ed. Peachtree Corners, GA: ASHRAE, 2021.

[3] M. Acton, P. Bertoldi and J. Booth, Best Practice Guidelines for the EU Code of Conduct on Data Centre Energy Efficiency, JRC Technical Note JRC136986, European Commission, Joint Research Centre, 2024.

[4] J. Stojkovic et al., "TAPAS: Thermal- and power-aware scheduling for LLM inference in cloud platforms," in Proc. 30th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25), 2025.

[5] Y. Zhang, H. Li and S. Wang, "The global energy impact of raising the space temperature for high-temperature data centers," Cell Reports Physical Science, vol. 4, no. 10, 2023.

[6] S. S. Rao, Engineering Optimization: Theory and Practice, 4th ed. Hoboken, NJ: Wiley, 2009.

[7] F. S. Hillier and G. J. Lieberman, Introduction to Operations Research, 10th ed. New York: McGraw-Hill, 2015.

[8] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261–272, 2020.

[9] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," Mathematical Programming Computation, vol. 10, no. 1, pp. 119–142, 2018.

[10] M. S. Andersen, J. Dahl and L. Vandenberghe, CVXOPT: A Python package for convex optimization, version 1.3. [Online]. Available: https://cvxopt.org

[11] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proc. 7th Python in Science Conference (SciPy 2008), pp. 11–15, 2008.

[12] F. P. Incropera, D. P. DeWitt, T. L. Bergman and A. S. Lavine, Fundamentals of Heat and Mass Transfer, 7th ed. Hoboken, NJ: Wiley, 2011.